A Bite of Distributed Communication

Khoa A. Vu & Kelly Pham

Student ID: 030063200 & 03039600

California State University, Long Beach

CECS 327 - Section 01

Dr. Hailu Xu

October 09, 2025

Objective

The purpose of this project is to build on our Docker foundations by:

1. Create Dockerized server and client applications
2. Demonstrate anycast (multiple servers, one chosen by the network)
3. Demonstrate multicast (one sender, multiple receivers)
4. Monitor traffic with tools like tcpdump or Wireshark inside containers or on host

Implementation

Anycast is where there are multiple servers. In this case all of our servers are using the same port of 5000 with their unique 'SERVER_NAME'. An initial challenge we had but was able to fix was differentiating our different servers when sending messages. By ensuring that our servers had their own unique name and including that in our docker-compose.yml, we were able to solve this issue. In our handle_client() function, this is used when a client connects to the server. From there, the server will print the message showing who connected as well as a message from the server "Hello from {name}". The run_server() function is used to establish the connections and listen for any clients by binding the host and port. For client.py, we are reading from the server and set the port to 5000 so it matches the server's listening port. We also created the TCP socket so it can connect with the server and decode the message it received from the server. With this, we can see in our output a client trying to connect to multiple servers with the "Hello from server" message.

Multicast had a different design as we needed multiple receivers this time and only one server. We had to create a UDP socket and we specified the multicast group to be 224.1.1.1 and port 5007 as instructed in the lab. We wrote a for loop for the message and used sock.sendto() to

ensure that we sent the message to the corresponding multicast_group and port. Finally, after the messages are sent out, we made sure to close the socket using sock.close(). Once again for multicast_receiver, we had to define the multicast_group and port. We also created a UDP socket and used sock.setsockopt() so multiple clients could connect to the same listening port. We have the line mrq = struct.pack('4sL', socket.inet_aton(GROUP), socket.INADDR_ANY) which indicates the client's intent and multicast request to join the multicast group (224.1.1.1). The next line includes socket.IP_ADD_MEMBERSHIP making the socket subscribe to the multicast group in order to receive the messages. From there we have message handling where it decodes the messages that it received from the server. The socket will close when the client chooses to leave the multicast group.

- Why UDP is not reliable
    - UDP is not reliable as it is connectionless and does not use the three way handshake as TCP does. As a result, data that is being transmitted may be lost or duplicated before the sender is able to access it. Additionally, there is no congestion control which can potentially lead to overwhelming the network.
- How group join/leave affect who receives packets
    - By including the line IP_ADD_MEMBERSHIP in the code, it allows a receiver to join the group. The receiver must be actively in the group by the time of the transmission to receive the data. If you leave the group, or close the socket (sock.close), the receiver will stop receiving any more data that is sent.
- Differences observed between single source and multi-source multicast

- Since single source multicast only has one sender, it is easier to manage and identify the source in which the message is being sent compared to a multi-source multicast.

Screenshots

Annycast

client.py

Project 2 > anycast > client.py > ...

```python
import os
import time

HOST = os.getenv('TARGET_HOST', 'server')  # Docker Compose service name
PORT = 5000

def client_once():
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
            s.settimeout(3)
            s.connect((HOST, PORT))
            data = s.recv(1024)
            print("Received:", data.decode('utf-8').strip())
    except Exception as e:
        print("Error:", e)

if __name__ == '__main__':
    for i in range(3):
        client_once()
        time.sleep(0.5)
```

client.py - connects to the server where it send a request and a response back

```python
import socket
import threading
import os

'''
Citation(s):
1) DigitalOcean. (2025, February 21). Python Socket Programming: Server and Client Example Guide. Retrieved from https://www.digitalocean.com/community/tutorials/python-socket-programming-server-
2) GeeksforGeeks – "How to Capture UDP Packets in Python" GeeksforGeeks. (2025, July 23). How to Capture UDP Packets in Python. Retrieved from https://www.geeksforgeeks.org/python/how-to-capture-
'''

# Configuration
HOST = '0.0.0.0'  # listen on all interfaces
PORT = 5000 # common port for custom python server

def handle_client(conn, addr, name):
    try:
        print(f"[{name}] connection from {addr}")
        msg = f"Hello from {name}\n".encode('utf-8')
        conn.sendall(msg)
    finally:
        conn.close()

def run_server(name):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        s.bind((HOST, PORT))
        s.listen()
        print(f"[{name}] listening on {HOST}:{PORT}")
        while True:
            conn, addr = s.accept()
            threading.Thread(target=handle_client, args=(conn, addr, name)).start()

if __name__ == '__main__':
    name = os.environ.get('SERVER_NAME', 'server')
    run_server(name)
```

server.py - listens for client connects, processes requests, sends request back to client

```dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY server.py /app/server.py
COPY client.py /app/client.py
CMD ["python", "server.py"]
```

Dockerfile

docker-compose.yml

Project 2 > anycast > docker-compose.yml

```yaml
version: '3'
Run All Services
services:
  Run Service
  server:
    build: .
    container_name: server
    environment:
      - SERVER_NAME=server
    # You can scale this service: docker compose up --scale server=3

  Run Service
  client:
    build: .
    container_name: client
    command: ["python", "client.py"]
    environment:
      - TARGET_HOST=server
```

docker-compose.yml - configures server and client services

Multicast

multicast_receiver.py

Project 2 > multicast > multicast_receiver.py > ...

```python

def receive(duration=10):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
    sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    sock.bind(('', PORT))

    mreq = struct.pack('4sL', socket.inet_aton(GROUP), socket.INADDR_ANY)
    sock.setsockopt(socket.IPPROTO_IP, socket.IP_ADD_MEMBERSHIP, mreq)
    print("Joined multicast group", GROUP, "port", PORT)

    sock.settimeout(1.0)
    end = time.time() + duration
    while time.time() < end:
        try:
            data, addr = sock.recvfrom(65536)
            try:
                decoded = data.decode('utf-8')
                print(f"Received from {addr}: {decoded}")
                try:
                    obj = json.loads(decoded)
                    print("  parsed json:", obj)
                except Exception:
                    pass
            except UnicodeDecodeError:
                print(f"Received binary from {addr}: len={len(data)}", data[:16])
        except socket.timeout:
            continue

    print("Leaving multicast group")
    sock.close()

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--duration', type=int, default=10)
    args = parser.parse_args()
    receive(duration=args.duration)

```

Multicast_receiver.py - listening for data from a multicast group while decoding messages received

multicast_sender.py

Project 2 > multicast > multicast_sender.py > ...

```python
import socket
import time
import json
import struct
import argparse

MULTICAST_GROUP = '224.1.1.1'
PORT = 5007

def send_messages(interval=1, count=5):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM, socket.IPPROTO_UDP)
    ttl = struct.pack('b', 1)
    sock.setsockopt(socket.IPPROTO_IP, socket.IP_MULTICAST_TTL, ttl)

    for i in range(count):
        msg = json.dumps({"sensor": "temp", "value": 20 + i}).encode('utf-8')
        sock.sendto(msg, (MULTICAST_GROUP, PORT))
        print("Sent JSON:", msg)
        time.sleep(interval)

    # Send binary data
    sock.sendto(b'\x00\x01\x02\x03', (MULTICAST_GROUP, PORT))
    print("Sent binary blob")
    sock.close()

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('--interval', type=float, default=1.0)
    parser.add_argument('--count', type=int, default=5)
    args = parser.parse_args()
    send_messages(interval=args.interval, count=args.count)
```

Multicast_sender.py - broadcasting data to all subscribed receivers on the network

docker-compose.yml

Project 2 > multicast > docker-compose.yml

```yaml
version: '3'
Run All Services
services:
  Run Service
  sender:
    build: .
    container_name: m_sender
    command: ["python", "multicast_sender.py", "--count", "10", "--interval", "0.5"]

  Run Service
  receiver1:
    build: .
    container_name: m_receiver1
    command: ["python", "multicast_receiver.py", "--duration", "15"]

  Run Service
  receiver2:
    build: .
    container_name: m_receiver2
    command: ["python", "multicast_receiver.py", "--duration", "15"]
```

docker-compose.yml - configures server and client services

Contributions

- Kelly Pham: Worked on and created the baseline of the files needed for the project, writing up the project report
- Khoa Vu: Tested the multiple files, Docker Compose setup, while also creating the README and video recording.
- Both: Reviewing task, documentation.

Result

video of code execution & output

References

1) DigitalOcean – "Python Socket Programming: Server and Client Example Guide" Kumar, Pankaj; Walia, Anish Singh. "Python Socket Programming: Server and Client Example Guide." DigitalOcean, updated February 21, 2025. Available at: https://www.digitalocean.com/community/tutorials/python-socket-programming-server-client
2) GeeksforGeeks – "How to Capture UDP Packets in Python" GeeksforGeeks. "How to Capture UDP Packets in Python." Last updated July 23, 2025. Available at: https://www.geeksforgeeks.org/python/how-to-capture-udp-packets-in-python/#